Experiment 2.1.1: Roots of a Quadratic Equation

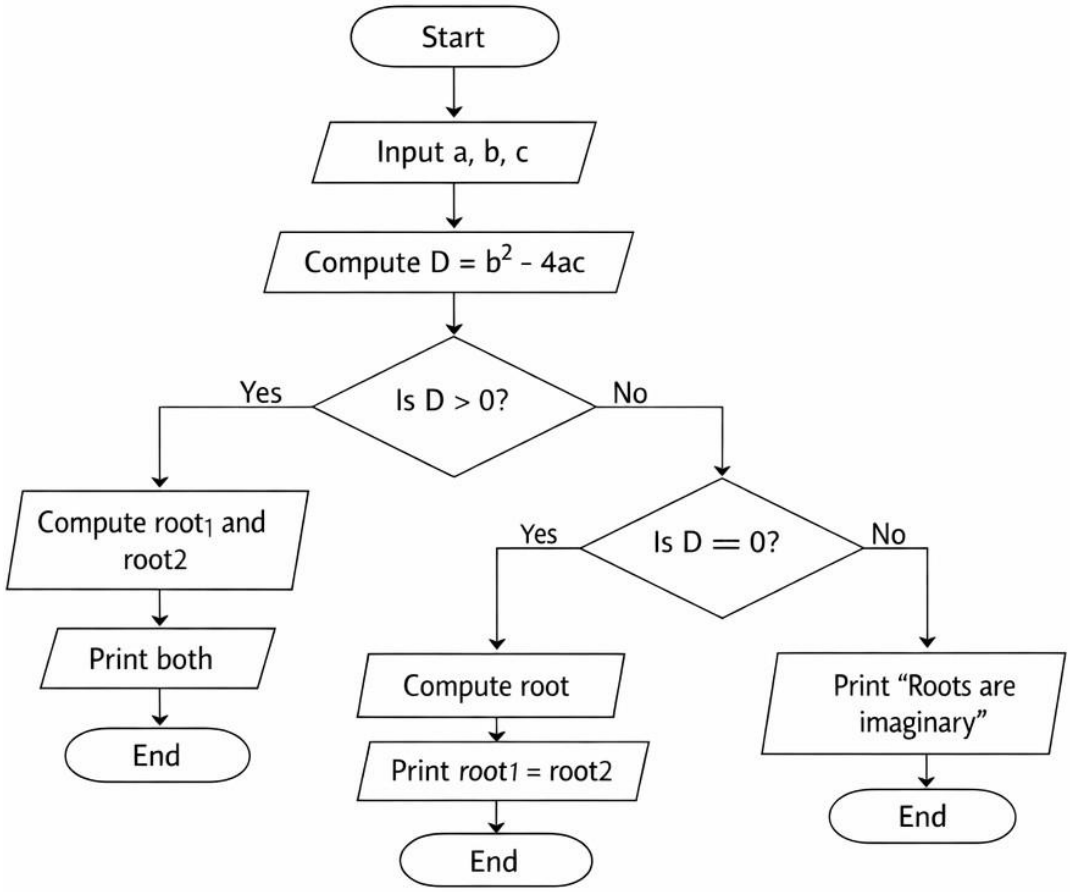
1. Aim

To design and implement a Python program that finds the roots of a quadratic equation $ax^2 + bx + c = 0$ using the quadratic formula. The program determines the nature of the roots (real and distinct, real and equal, or imaginary) based on the discriminant $D = b^2 - 4ac$ and formats the results to two decimal places.

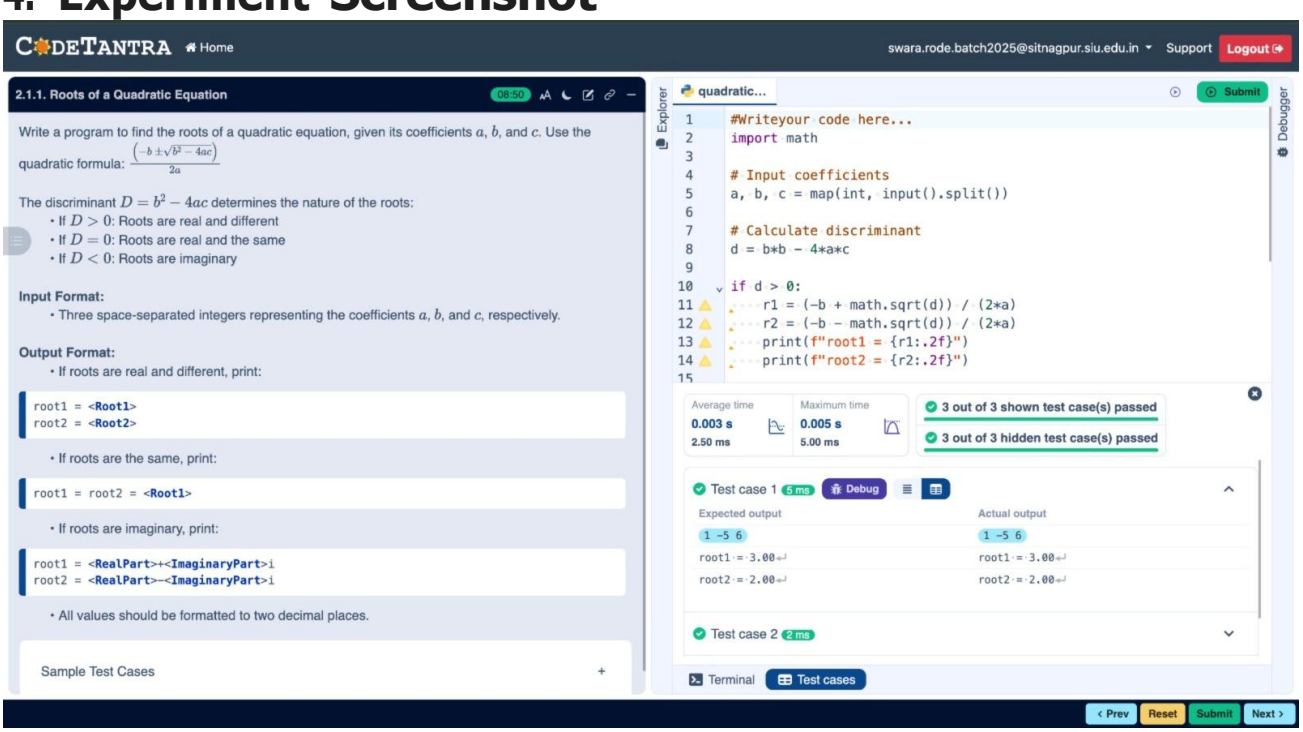
2. Pseudocode

1. **START**
2. READ three space-separated integers a, b, and c.
3. **CALCULATE** the discriminant: $D = b^2 - 4ac$.
4. **IF** $D > 0$:
   - Calculate $root1 = \frac{-b+\sqrt{D}}{2a}$
   - Calculate $root2 = \frac{-b-\sqrt{D}}{2a}$
   - **PRINT** "root1 = {root1}" and "root2 = {root2}" formatted to 2 decimal places.
5. **ELSE IF** $D == 0$:
   - Calculate $root = \frac{-b}{2a}$
   - **PRINT** "root1 = root2 = {root}" formatted to 2 decimal places.
6. **ELSE** (if $D < 0$):
   - **PRINT** "Roots are imaginary".
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 5.1.1: Leap Year Checker

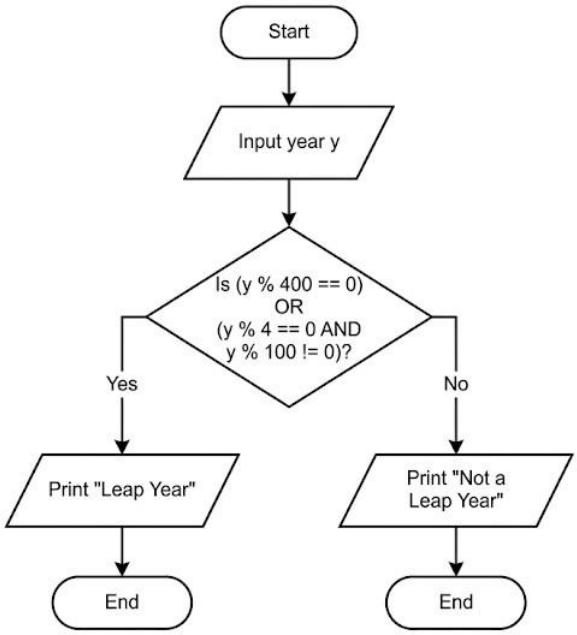
1. Aim

To design and implement a Python program that prompts the user to enter a year and determines whether the given year is a leap year or not, printing the appropriate message.

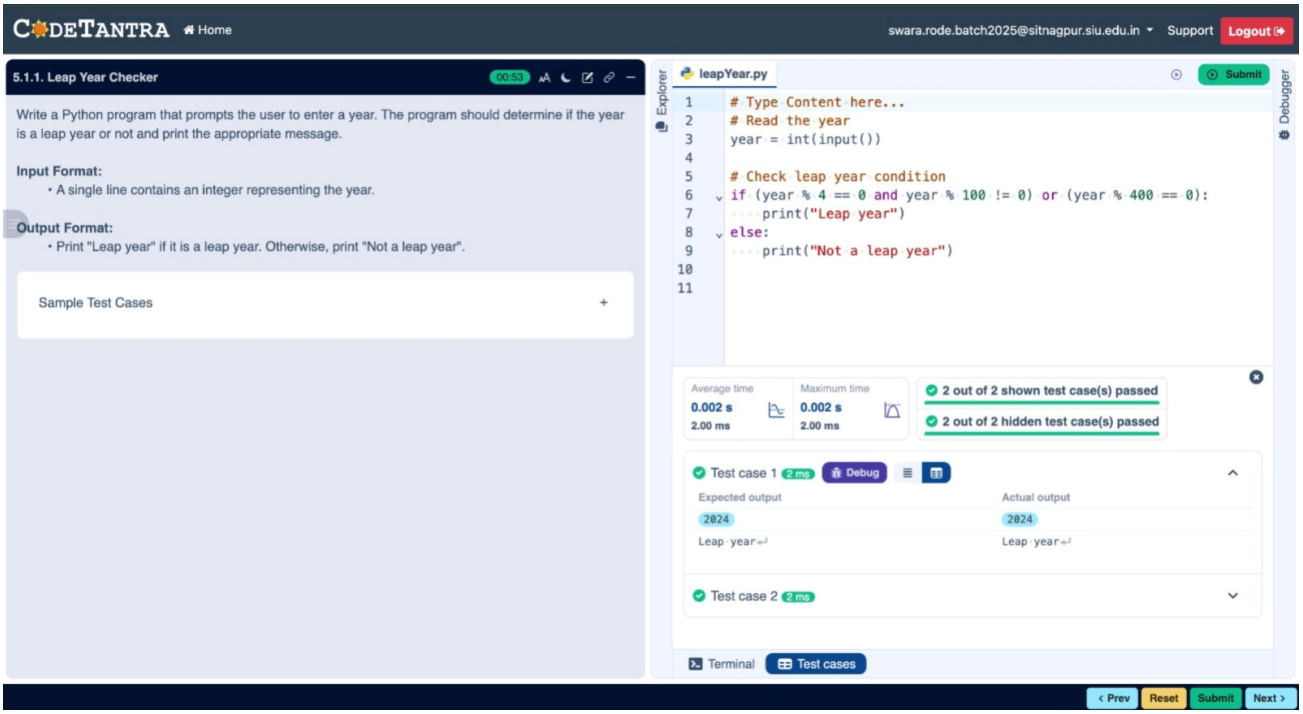
2. Pseudocode

1. **START**
2. **READ** the input value from the user and convert it to an integer.
3. **STORE** this value in the variable y.
4. **CHECK** if the year y is divisible by 400 (y % 400 == 0) **OR** check if the year y is divisible by 4 **AND** not divisible by 100 (y % 4 == 0 and y % 100 != 0).
5. **IF** the condition in step 4 is True, **PRINT** "Leap year".
6. **ELSE**, **PRINT** "Not a leap year".
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 5.1.2: Student Grade Based on Aggregate

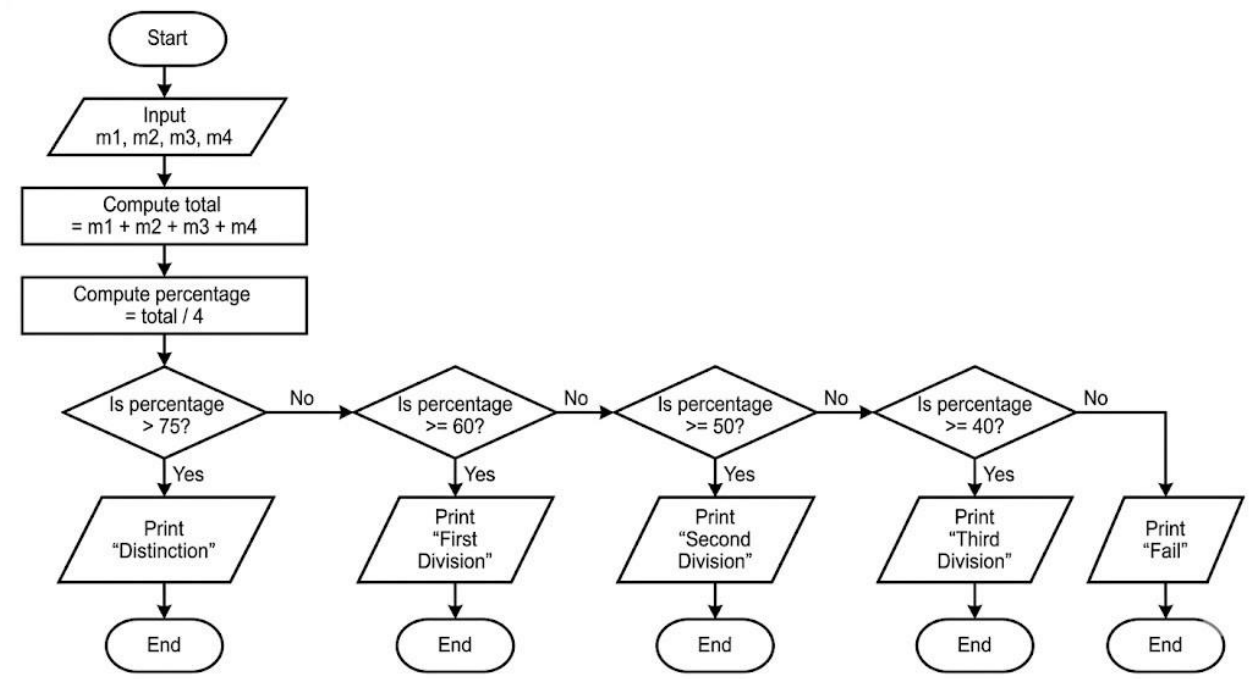
1. Aim

To design and implement a Python program that calculates a student's total marks, aggregate percentage, and final grade based on the marks obtained in four subjects. The program determines the grade using specific percentage-based conditions.

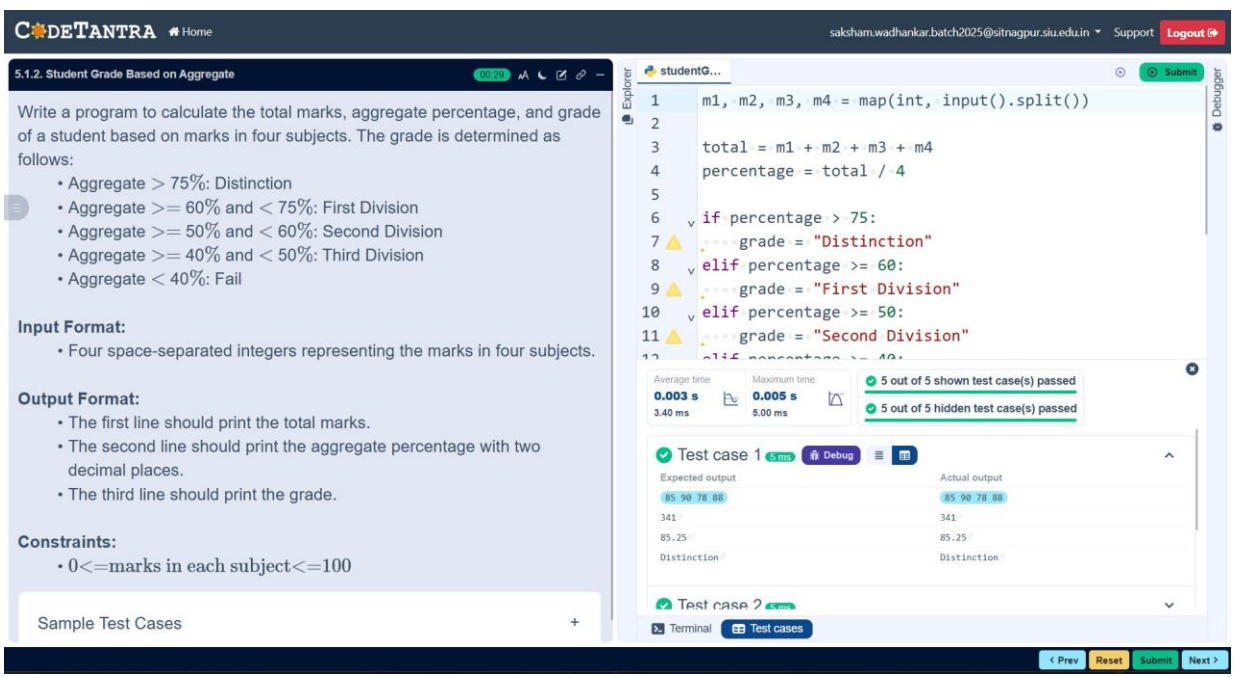
2. Pseudocode

1. **START**
2. **READ** four space-separated integers from the user and **STORE** them in variables m1, m2, m3, and m4.
3. **CALCULATE** the sum of the four subjects and **STORE** it in the variable total ($total = m1 + m2 + m3 + m4$).
4. **CALCULATE** the aggregate percentage by dividing the total by 4 and **STORE** it in the variable percentage.
5. **CHECK** the value of percentage to determine the grade:
   - **IF** percentage > 75, **STORE** "Distinction" in grade.
   - **ELSE IF** percentage >= 60, **STORE** "First Division" in grade.
   - **ELSE IF** percentage >= 50, **STORE** "Second Division" in grade.
   - **ELSE IF** percentage >= 40, **STORE** "Third Division" in grade.
   - **ELSE**, **STORE** "Fail" in grade.
6. **PRINT** the total marks.
7. **PRINT** the percentage formatted to two decimal places.
8. **PRINT** the grade.
9. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 6.1.1: Incremented Date

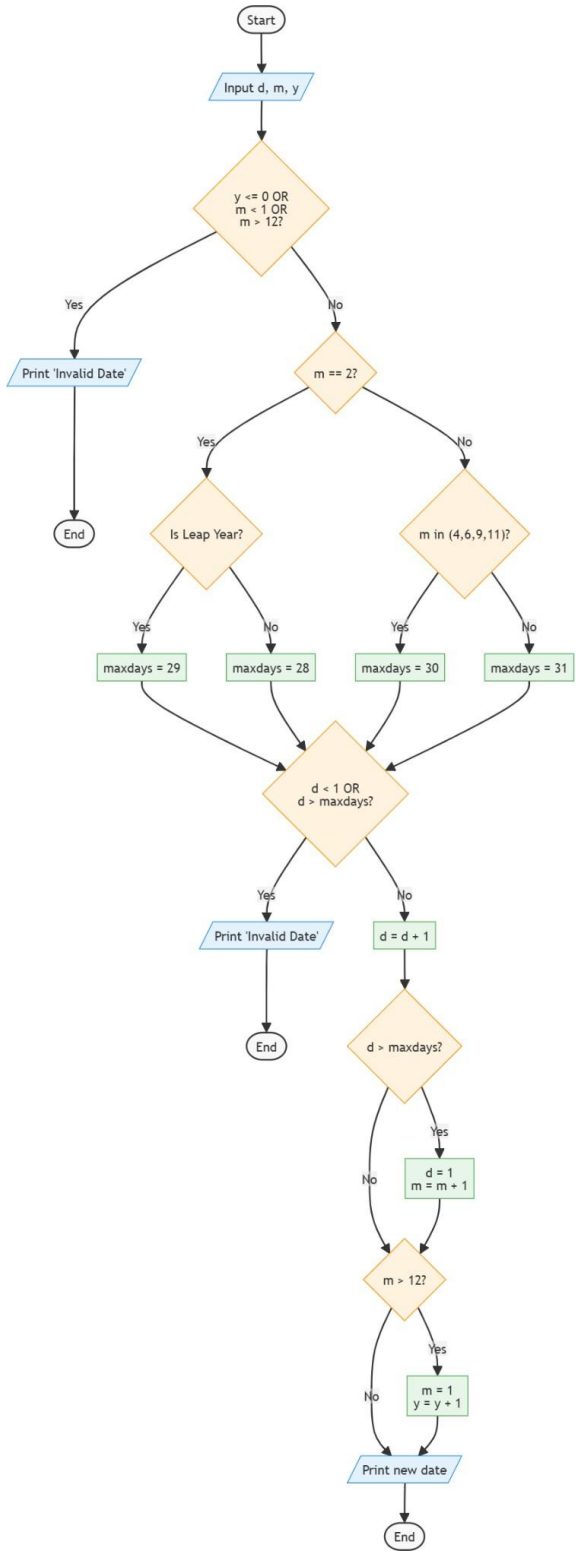
1. Aim

To design and implement a Python program that checks if a given date (day, month, year) is valid according to the Gregorian calendar rules. If the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY format. If the date is invalid, it prints "Invalid Date".

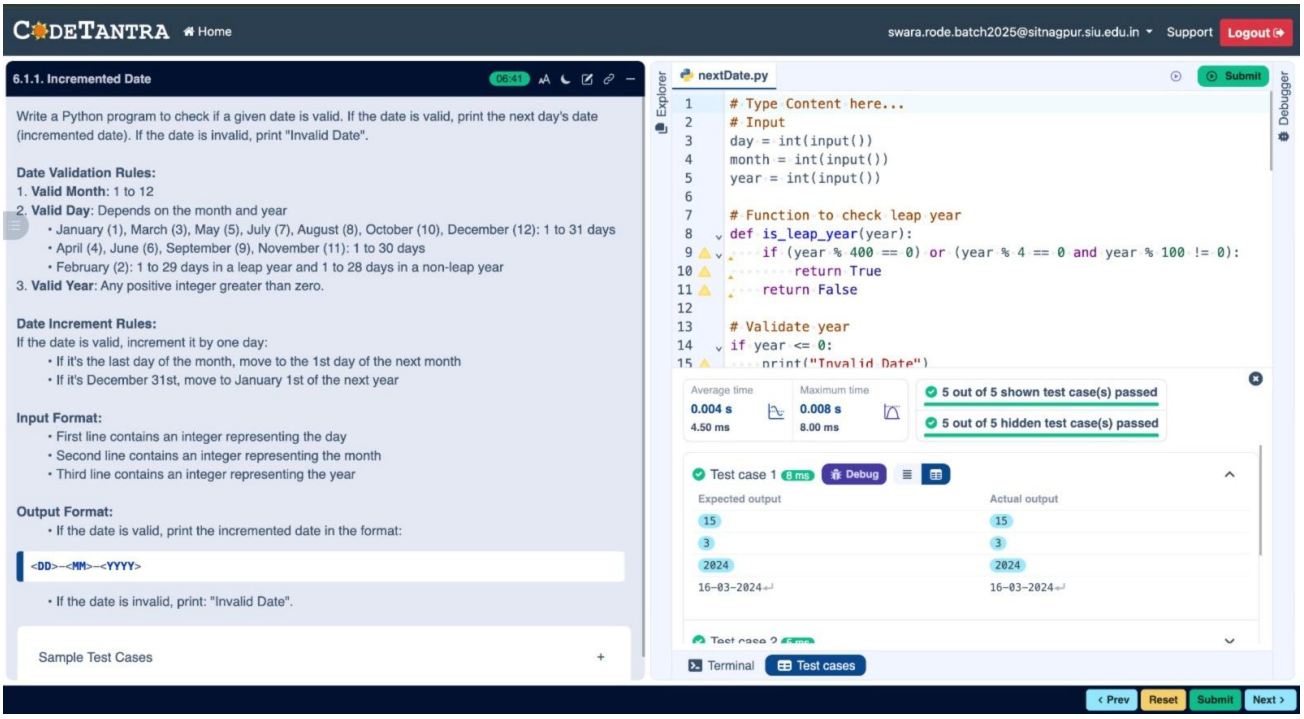
2. Pseudocode

1. **START**
2. **READ** three integers representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True.
4. **CHECK** basic validity: **IF** $y \leq 0$ **OR** $m < 1$ **OR** $m > 12$, **SET** flag to False.
5. **IF** flag is True:
   - **DETERMINE** maxdays based on the month:
     - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
     - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
     - **ELSE IF** m is 2 (February):
       - **IF** the year y is a leap year: maxdays = 29.
       - **ELSE**: maxdays = 28.
   - **CHECK** day validity: **IF** $d < 1$ **OR** $d > maxdays$, **SET** flag to False.
6. **IF** flag is True:
   - **INCREMENT** the day d by 1.
   - **IF** d > maxdays (End of month):
     - **SET** d = 1.
     - **INCREMENT** the month m by 1.
     - **IF** m > 12 (End of year):
       - **SET** m = 1.
       - **INCREMENT** the year y by 1.
   - **PRINT** the incremented date formatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
   - **PRINT** "Invalid Date".
8. **END**

3. Flowchart


Start
Input d, m, y
y <= 0 OR
m < 1 OR
m > 12?
Yes
No
Print 'Invalid Date'
End
m == 2?
Yes
No
Is Leap Year?
m in (4,6,9,11)?
Yes
No
Yes
No
maxdays = 29
maxdays = 28
maxdays = 30
maxdays = 31
d < 1 OR
d > maxdays?
Yes
No
Print 'Invalid Date'
d = d + 1
End
d > maxdays?
Yes
No
d = 1
m = m + 1
m > 12?
Yes
No
m = 1
y = y + 1
Print new date
End

4. Experiment Screenshot



Experiment 6.1.2: Factorial of a Number

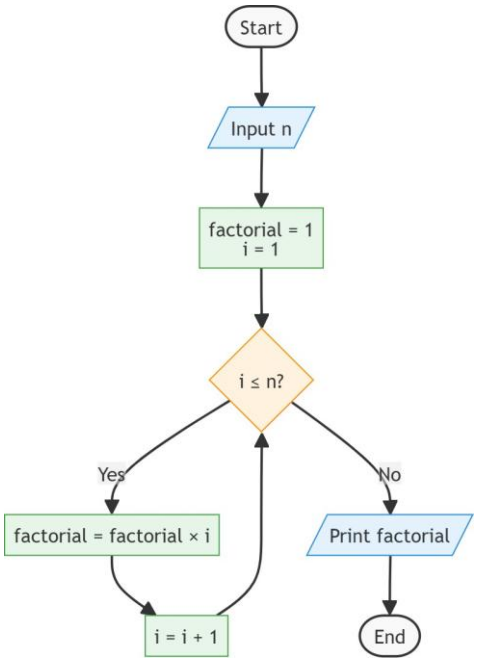
1. Aim

To design and implement a Python program that calculates the factorial of a given integer n using a for loop. The program reads the integer, computes its factorial iteratively, and prints the final result.

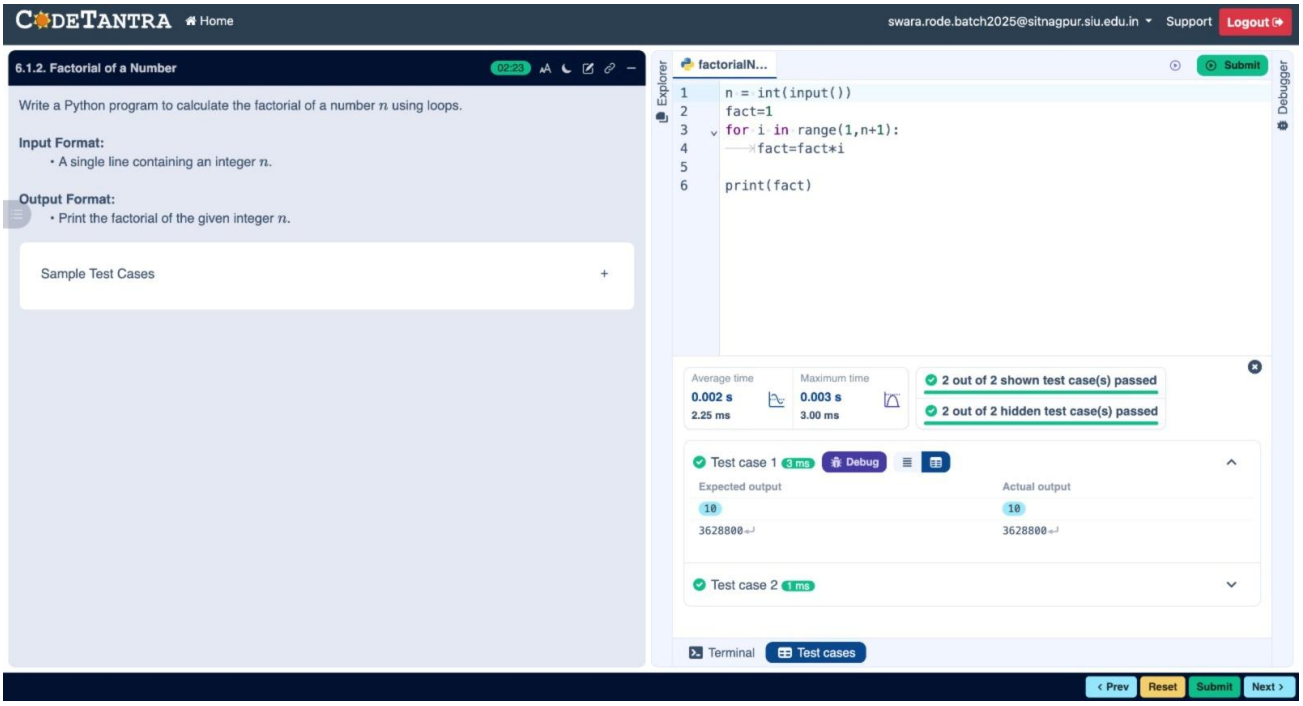
2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n.
3. **INITIALIZE** a variable factorial to 1.
4. **FOR** each number i in the range from 1 to n (inclusive):
    - **MULTIPLY** factorial by i ($factorial = factorial * i$).
    - **STORE** the new value back into factorial.
5. **END FOR** loop once all numbers up to n have been multiplied.
6. **PRINT** the final value of factorial.
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 7.1.1: Inverted Star Pattern

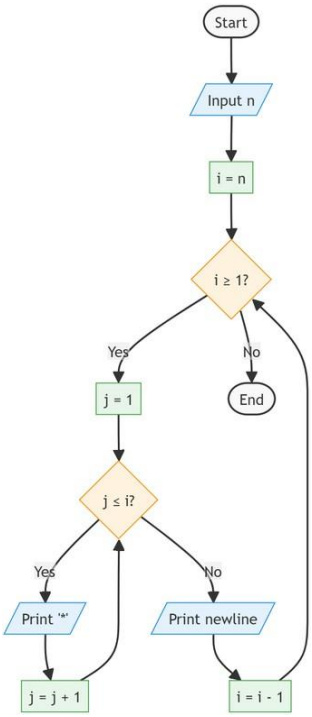
1. Aim

To design and implement a Python program that prints an inverted right-angled triangle star pattern. For a given integer n, the program uses nested loops to print n rows of stars, starting with n stars in the first row and decreasing by one star in each subsequent row until the last row has 1 star.

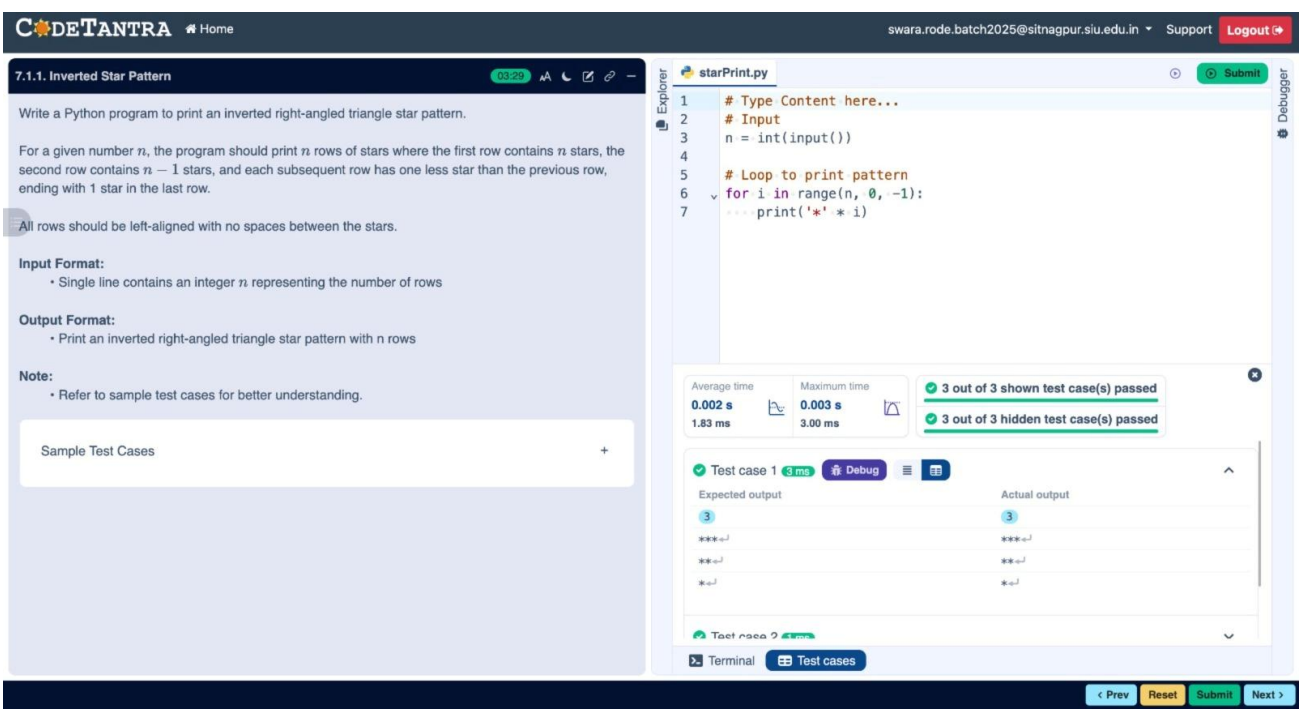
2. Pseudocode

1. **START**
2. **READ** a single integer from the user and **STORE** it in the variable n.
3. **FOR** each number i in the range starting from n down to 1 (decrementing by 1):
    - **FOR** each number j in the range from 0 to $i - 1$:
        - **PRINT** a star "*" without a newline character at the end.
    - **END INNER FOR**
    - **PRINT** a newline character to move to the next row.
4. **END OUTER FOR**
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 7.1.2: Possible Combinations firom Three Digits

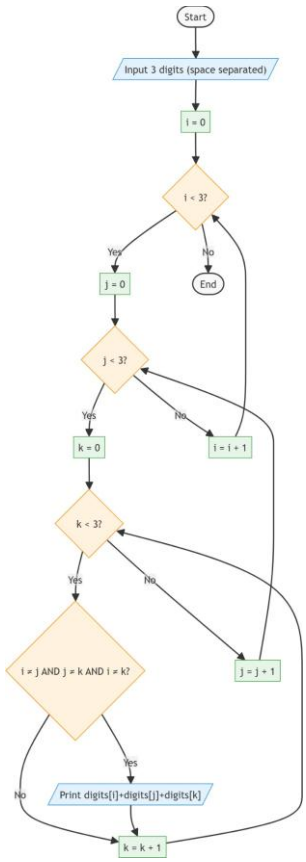
1. Aim

To design and implement a Python program that accepts three single-digit inputs (separated by spaces) and prints all six possible 3-digit combinations of these digits. Each digit should be used exactly once in each combination. The program maintains leading zeros if they are present.

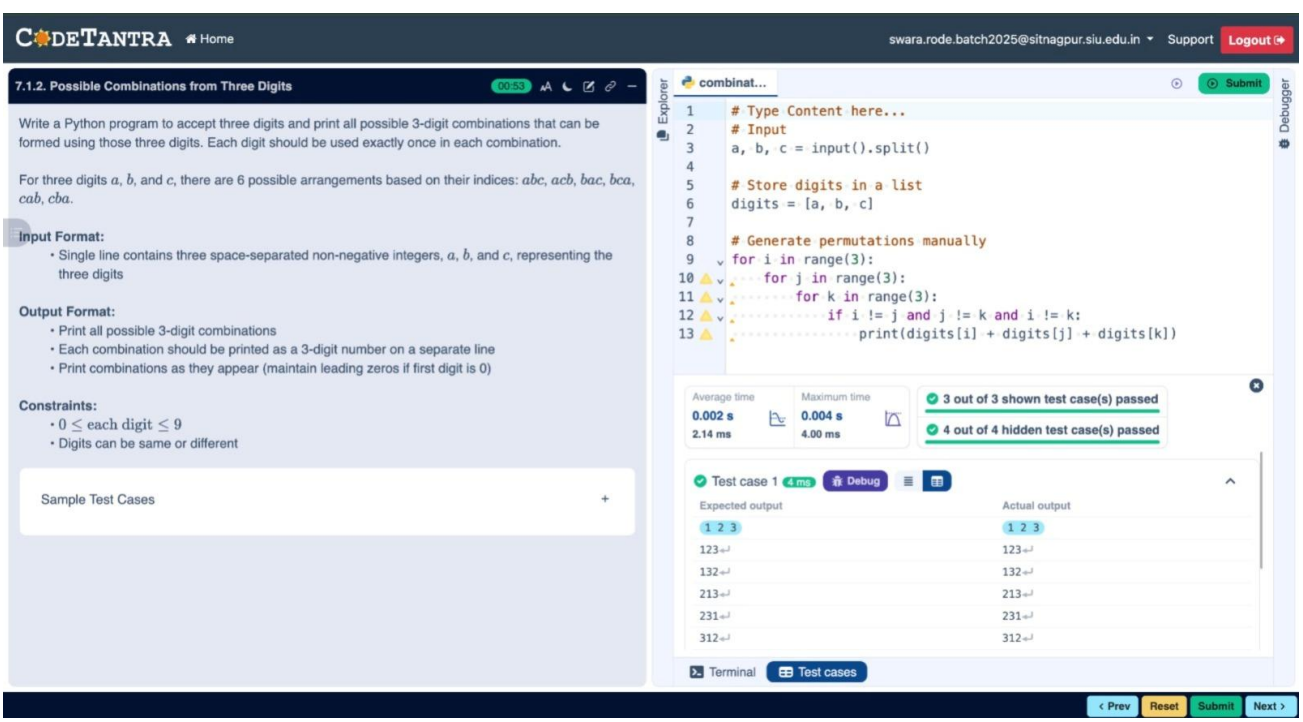
2. Pseudocode

1. **START**
2. **READ** a single line of input, split it by spaces, and **STORE** the three values into a list named digits.
3. **FOR** each index i from 0 to 2 (Outer Loop):
    - **FOR** each index j from 0 to 2 (Middle Loop):
        - **FOR** each index k from 0 to 2 (Inner Loop):
            - **CHECK IF** i is not equal to j **AND** j is not equal to k **AND** i is not equal to k.
            - **IF** the condition is True:
                - **PRINT** the concatenation of digits[i], digits[j], and digits[k].
4. **END LOOPS**
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 8.1.1: Multiplication ofi Two Square Matrices

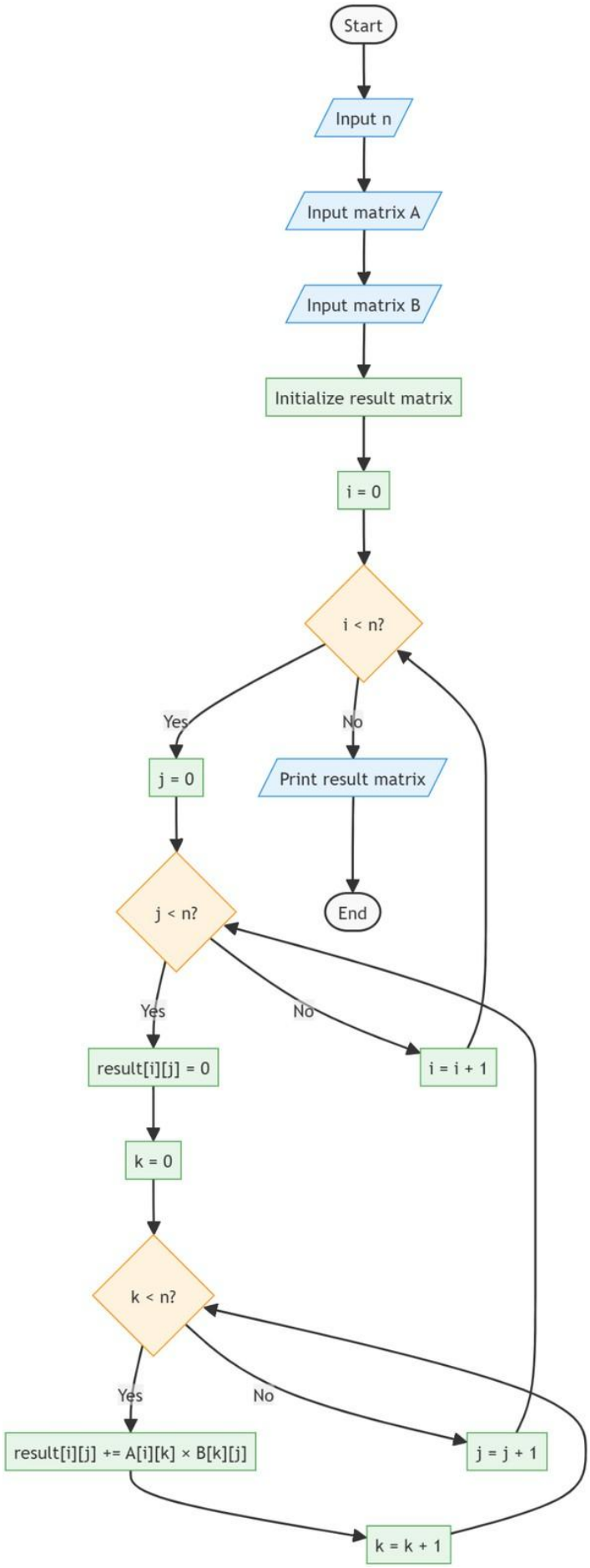
1. Aim

To design and implement a Python program that performs the multiplication of two square matrices of dimension $n \times n$. The program takes the dimension and the elements of both matrices as input from the user and computes their product.

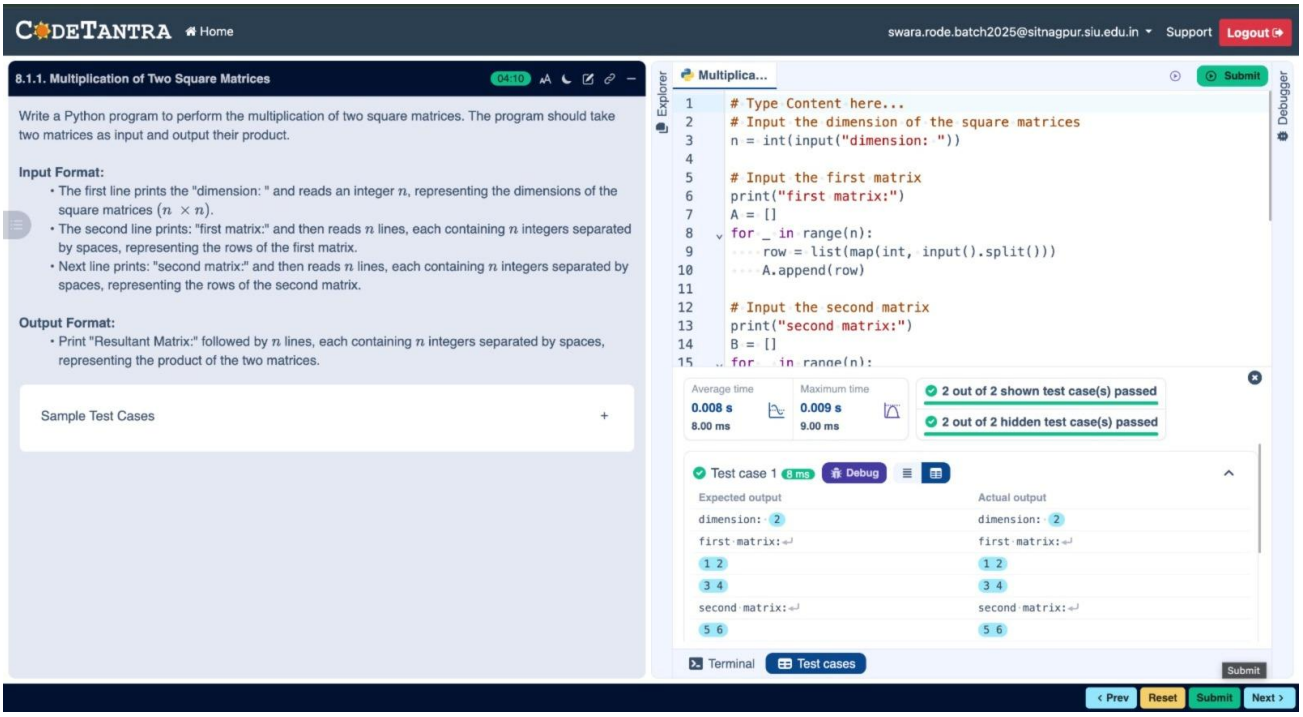
2. Pseudocode

1. **START**
2. **PRINT** "dimension: " and **READ** an integer from the user, storing it in the variable n.
3. **PRINT** "first matrix:" and **INITIALIZE** an empty list A.
4. **FOR** each row i from 0 to $n-1$:
    - **READ** space-separated integers from the user, convert them into a list, and append to A.
5. **PRINT** "second matrix:" and **INITIALIZE** an empty list B.
6. **FOR** each row i from 0 to $n-1$:
    - **READ** space-separated integers from the user, convert them into a list, and append to B.
7. **INITIALIZE** a result matrix of size $n \times n$ filled with zeros.
8. **FOR** i in range n (iterating through rows of A):
    - **FOR** j in range n (iterating through columns of B):
        - **FOR** k in range n (iterating through rows of B):
            - **ADD** the product of A[i][k] and B[k][j] to result[i][j].
9. **PRINT** "Resultant Matrix:".
10. **FOR** each row r in the result matrix:
    - **PRINT** the elements of r separated by spaces.
11. **END**

3. Flowchart


Start
Input n
Input matrix A
Input matrix B
Initialize result matrix
i = 0
i < n?
Yes
No
j = 0
Print result matrix
End
j < n?
Yes
No
result[i][j] = 0
i = i + 1
k = 0
k < n?
Yes
No
result[i][j] += A[i][k] × B[k][j]
j = j + 1
k = k + 1

4. Experiment Screenshot



Experiment 8.1.2: Prime Numbers in a Given Range

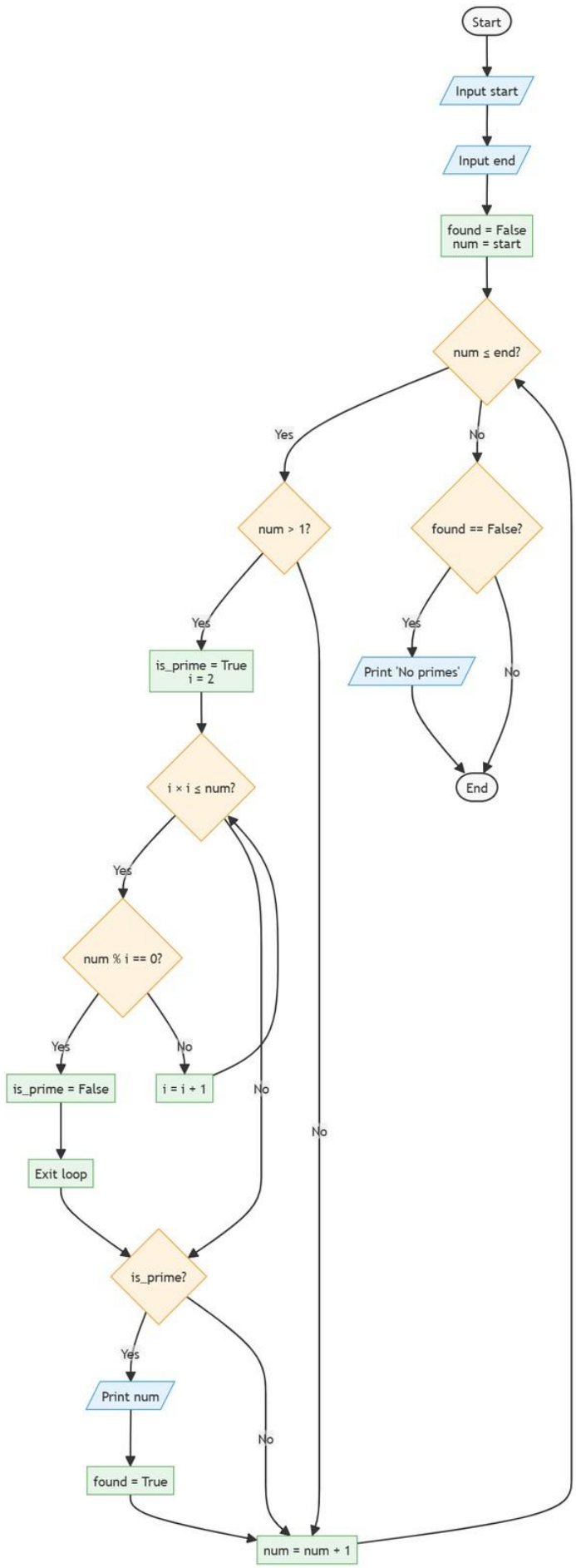
1. Aim

To design and implement a Python program that finds and prints all prime numbers within a specified range [start, end] (inclusive). The program determines the primality of each number and prints it. If no prime numbers exist within the given range, the program prints "No primes".

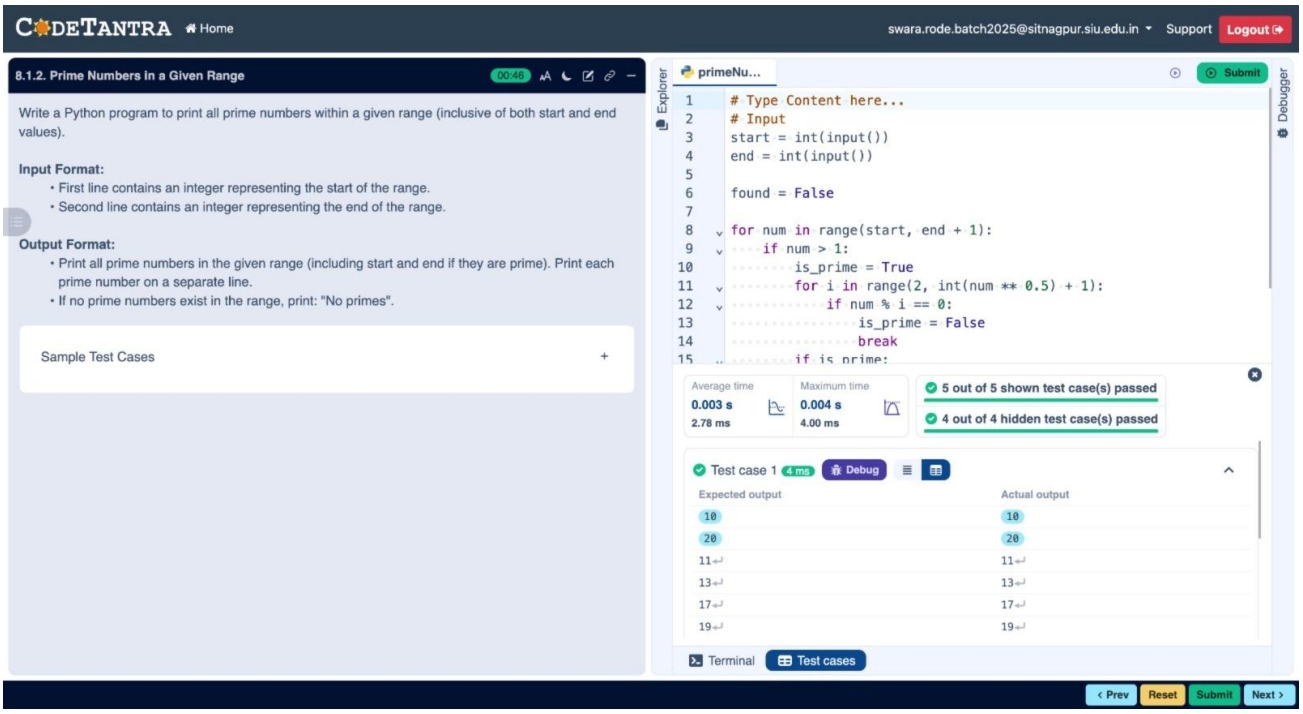
2. Pseudocode

1. **START**
2. **READ** an integer representing the start of the range and **STORE** it in start.
3. **READ** an integer representing the end of the range and **STORE** it in end.
4. **INITIALIZE** a boolean variable found to False.
5. **FOR** each number num in the range from start to end (inclusive):
    - **IF** num > 1:
        - **INITIALIZE** is_prime to True.
        - **FOR** each number i from 2 up to the integer square root of num ($int(num^{0.5}) + 1$):
            - **IF** num is divisible by i (num % i == 0):
                - **SET** is_prime to False.
                - **BREAK** the inner loop.
        - **IF** is_prime is True:
            - **PRINT** the prime number num.
            - **SET** found to True.
6. **IF** found is False:
    - **PRINT** "No primes".
7. **END**

3. Flowchart


Start
Input start
Input end
found = False
num = start
num ≤ end?
Yes
No
num > 1?
found == False?
Yes
Print 'No primes'
No
End
Yes
is_prime = True
i = 2
i × i ≤ num?
Yes
num % i == 0?
Yes
No
is_prime = False
i = i + 1
No
Exit loop
No
is_prime?
Yes
Print num
No
found = True
num = num + 1

4. Experiment Screenshot



Experiment 9.1.1: Count Vowels in a String - 1

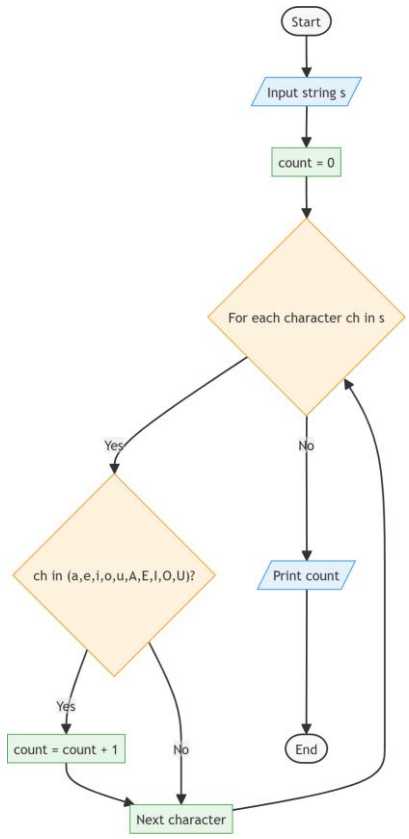
1. Aim

To design and implement a Python program that takes a string as input and prints the total count of vowels present in that string. The program should consider both uppercase and lowercase vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U').

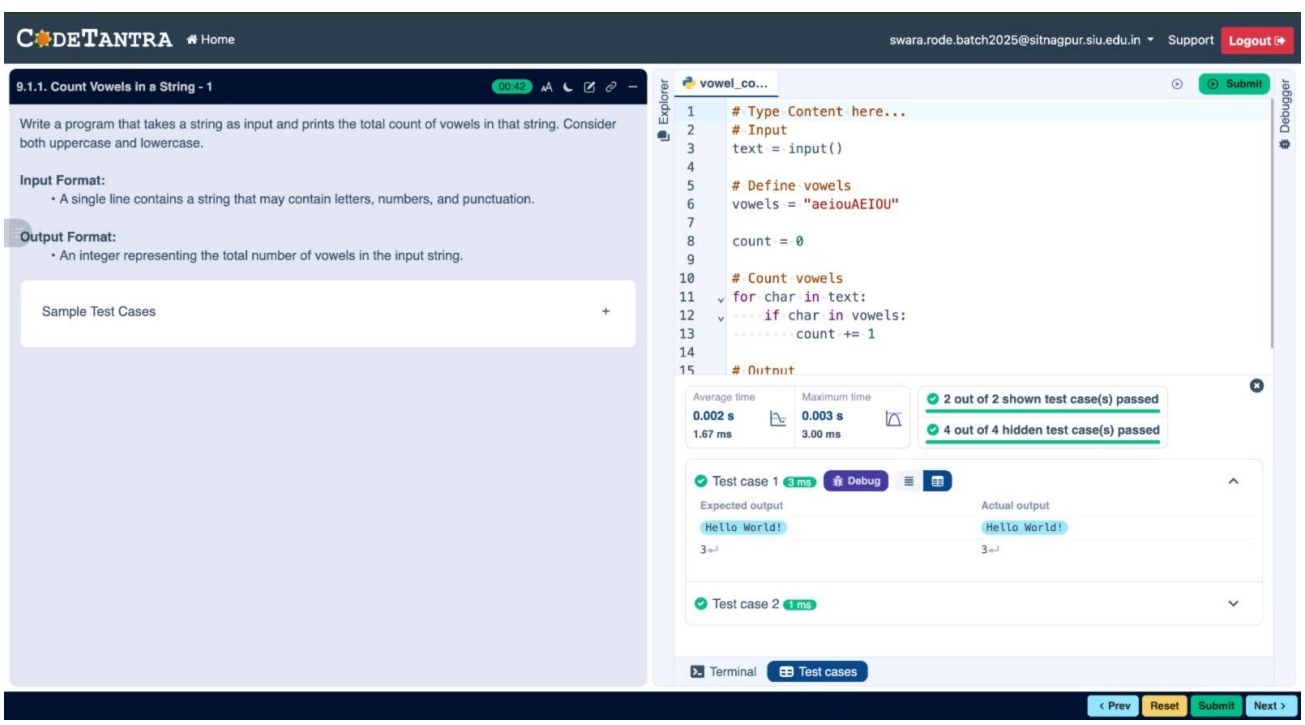
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **INITIALIZE** a variable count to 0 to keep track of the number of vowels.
4. **FOR** each character ch in the string s:
    - **CHECK IF** the character ch is present in the set of vowels "aeiouAEIOU".
    - **IF** the condition is True, **INCREMENT** count by 1.
5. **PRINT** the final value of count.
6. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 9.1.2: Palindrome or Not

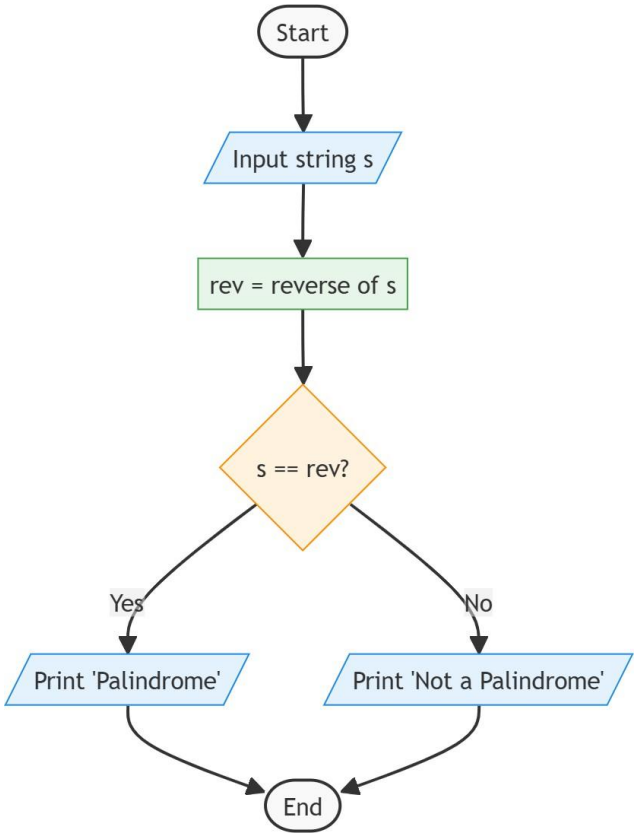
1. Aim

To design and implement a Python program that takes a string as input and checks whether the given string is a palindrome or not. A palindrome is a string that reads the same forwards and backwards.

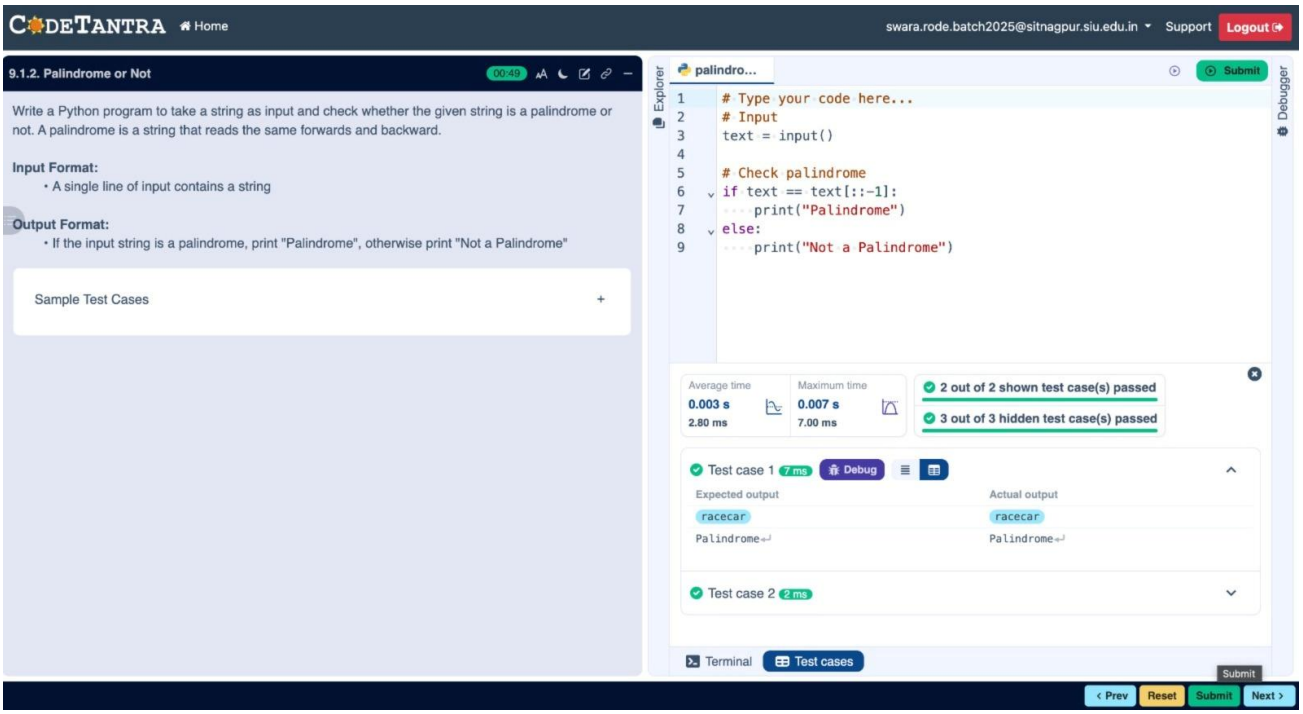
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **CREATE** a reversed version of the string s using slicing (s[::-1]) and **STORE** it in the variable rev.
4. **CHECK IF** the original string s is equal to the reversed string rev.
   - **IF** True, **PRINT** "Palindrome".
   - **ELSE**, **PRINT** "Not a Palindrome".
5. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 10.1.1: Remove Punctuations firom a String

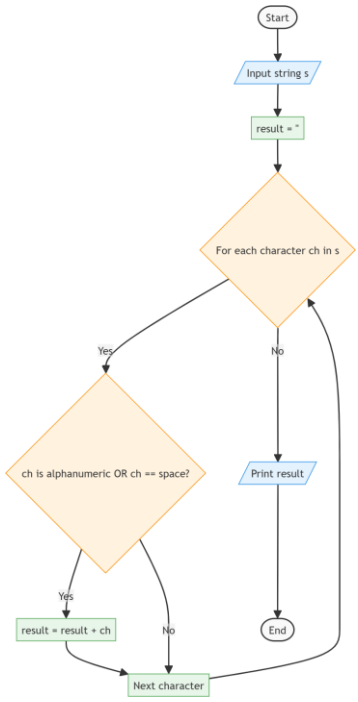
1. Aim

To design and implement a Python program that removes all punctuation marks from a given input string. Punctuation marks are defined as special characters that are not letters (uppercase or lowercase), digits, or spaces. The program should preserve alphanumeric characters and spaces while printing the final cleaned string.

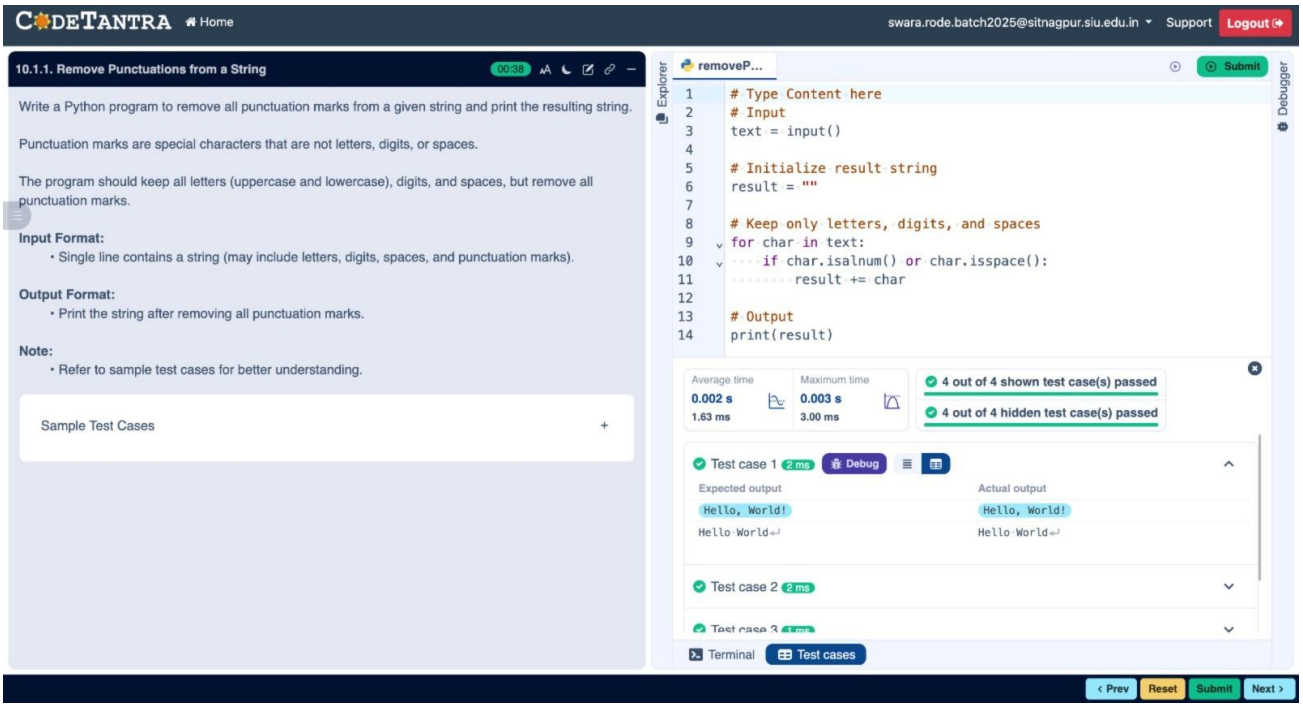
2. Pseudocode

1. **START**
2. **READ** the input string from the user and **STORE** it in the variable s.
3. **INITIALIZE** an empty string variable named result to store the cleaned text.
4. **FOR** each character ch in the input string s:
    - **CHECK IF** ch is an alphanumeric character (letter or digit) **OR** if ch is a space (" ").
    - **IF** the condition is True, **APPEND** the character ch to the result string.
5. **PRINT** the final value of the result string.
6. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 10.1.2: Reverse a String

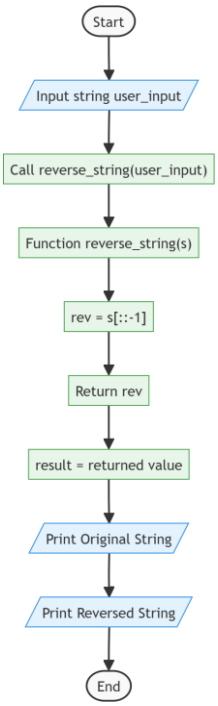
1. Aim

To design and implement a Python program that prints the reverse of a given input string. The program should use a user-defined function named reverse_string() that takes a string as input and returns its reversed version using string slicing.

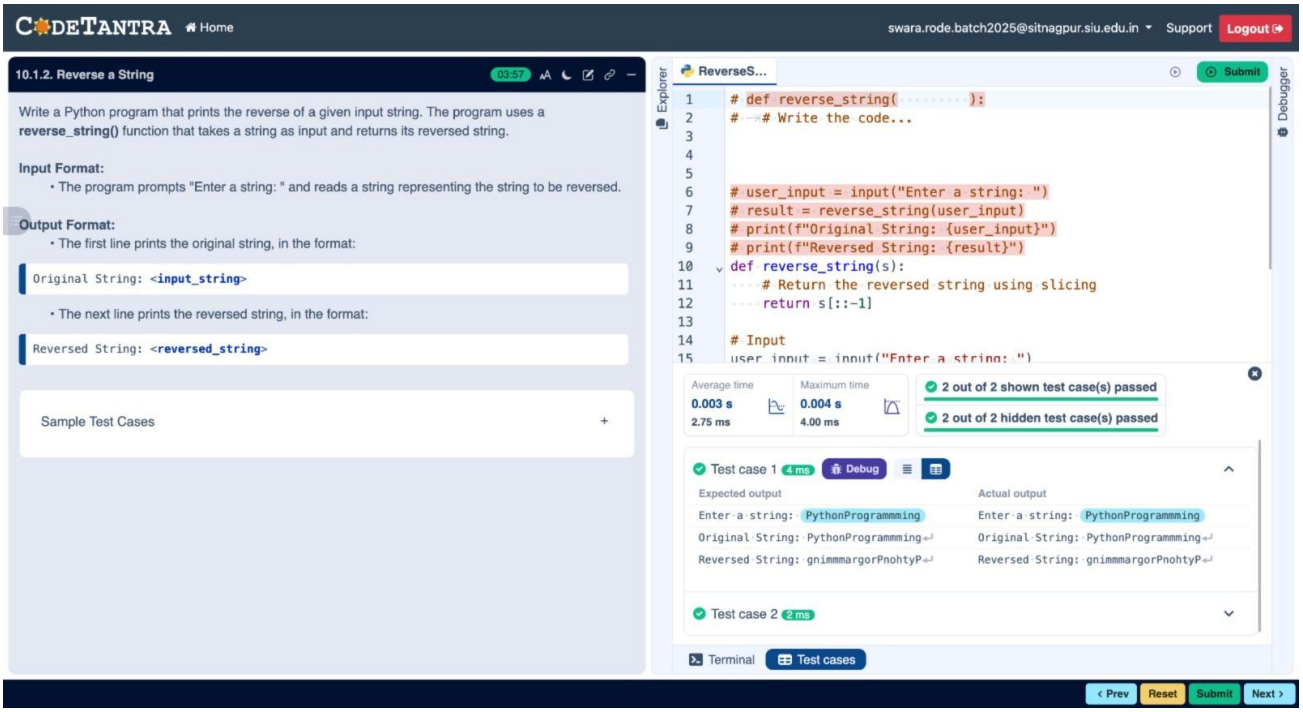
2. Pseudocode

1. **START**
2. **DEFINE** a function reverse_string(s):
   - **RETURN** the reversed string using slicing: s[::-1].
3. **READ** the input string from the user with the prompt "Enter a string: " and **STORE** it in the variable user_input.
4. **CALL** the reverse_string() function with user_input as an argument and **STORE** the returned value in the variable result.
5. **PRINT** the original string in the format: "Original String: {user_input}".
6. **PRINT** the reversed string in the format: "Reversed String: {result}".
7. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 11.1.1: Sum ofi Digits ofi a Number

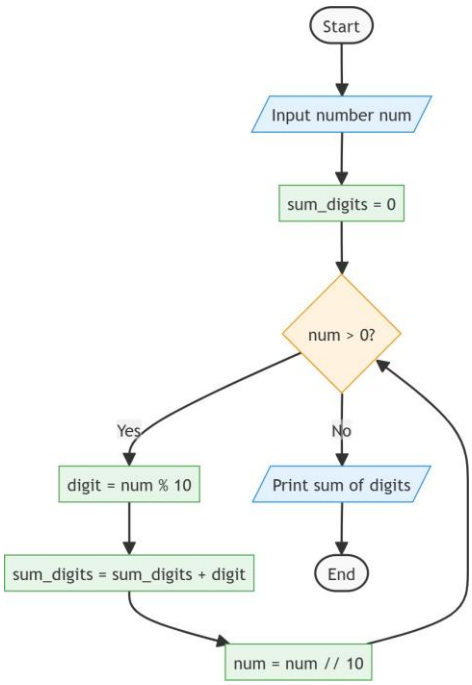
1. Aim

To design and implement a Python program that reads an integer from the user, calculates the sum of its individual digits using a loop, and prints the final result.

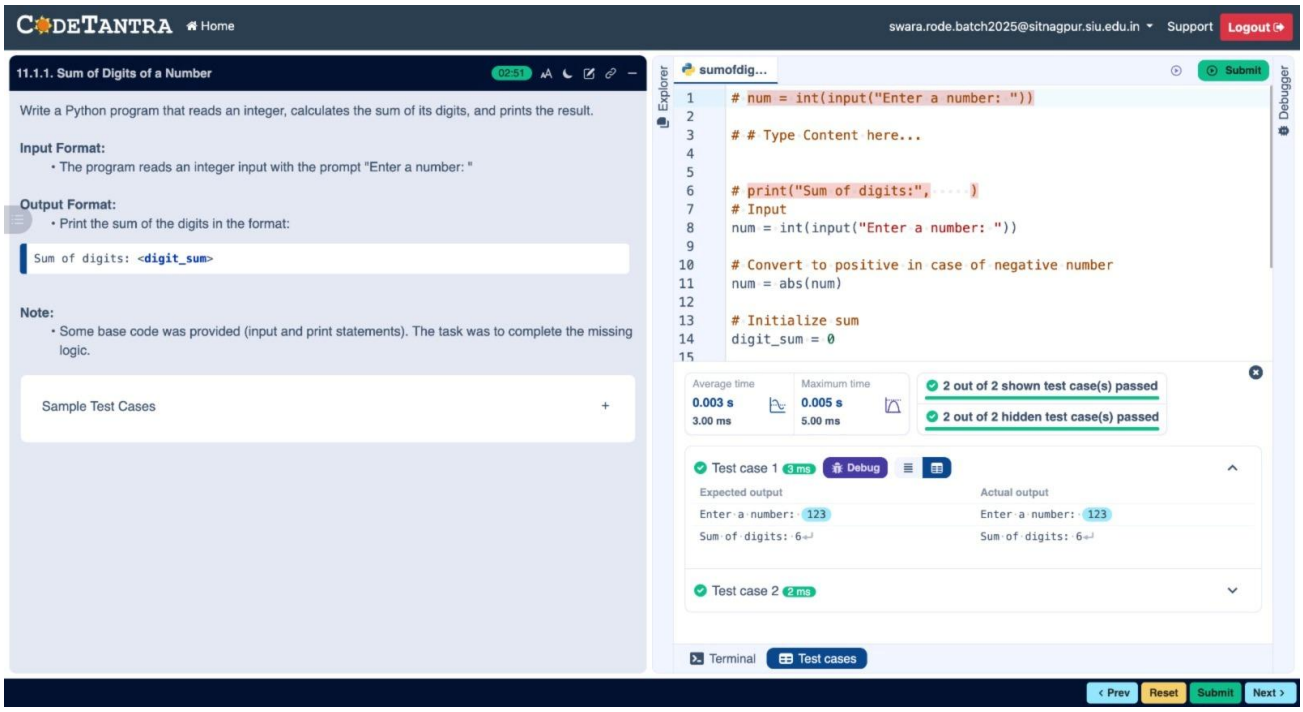
2. Pseudocode

1. **START**
2. **READ** an integer input from the user with the prompt "Enter a number: " and **STORE** it in the variable num.
3. **INITIALIZE** a variable sum_digits to 0 to accumulate the sum.
4. **WHILE** num is greater than 0:
    - **EXTRACT** the last digit of num using the modulo operator: digit = num % 10.
    - **ADD** the digit to sum_digits.
    - **REMOVE** the last digit from num using integer division: num = num // 10.
5. **PRINT** the result in the format: "Sum of digits: {sum_digits}".
6. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 11.1.2: Sum of Digits using Recursion

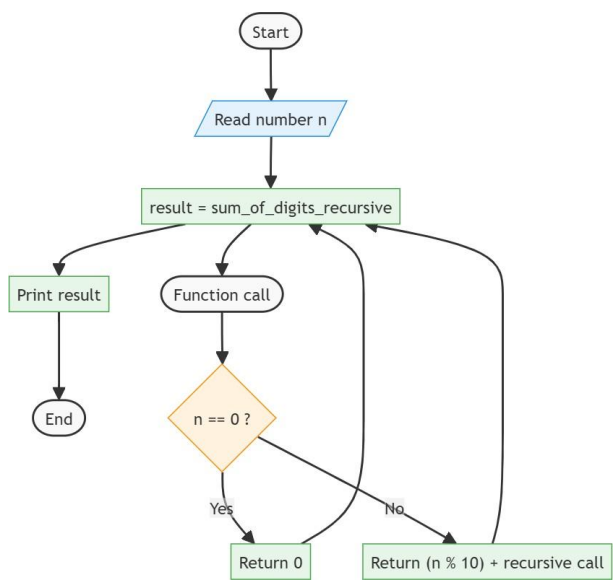
1. Aim

To design and implement a Python program that calculates the sum of the digits of a given non-negative integer n using a recursive function sum_of_digits_recursive(n).

2. Pseudocode

1. **START**
2. **DEFINE** a recursive function sum_of_digits_recursive(n):
   - **IF** $n == 0$, **RETURN** 0 (Base case).
   - **ELSE**:
     - **EXTRACT** the last digit: $n\%10$.
     - **RETURN** the last digit added to the result of a recursive call with the remaining digits: sum_of_digits_recursive(n // 10).
3. **READ** a non-negative integer from the user and **STORE** it in the variable number.
4. **CALL** sum_of_digits_recursive(number) and **STORE** the result in result.
5. **PRINT** the result.
6. **END**

3. Flowchart



4. Experiment Screenshot

CODETANTRA Home

swara.rode.batch2025@sitnagpur.siu.edu.in Support Logout

11.1.2. Sum of Digits using Recursion

04:43

Write a recursive function **sum_of_digits_recursive(n)** that calculates the sum of the digits of a given non-negative integer n.

Input Format:

- The input consists of a single non-negative integer n.

Output Format:

- The output should be the sum of the digits of the non-negative integer n.

Constraints:

- $0 \leq n \leq 10^9$

Sample Test Cases +

Explorer

SumofDi...

Submit

Debugger

```
# def sum_of_digits_recursive(n):

#     # Write your code here

#     return     sum_of_digits_recursive(   )



# # Write your code here

# result = sum_of_digits_recursive(number)


    # Recursive function to calculate sum of digits
def sum_of_digits_recursive(n):
```

Average time	Maximum time
0.002 s	0.002 s
1.80 ms	2.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 2 ms Debug

Expected output	Actual output
23	23
5	5

Test case 2 2 ms

Terminal Test cases

Submit

Prev Reset Submit Next

Experiment 12.1.1: Phone Number Database Management

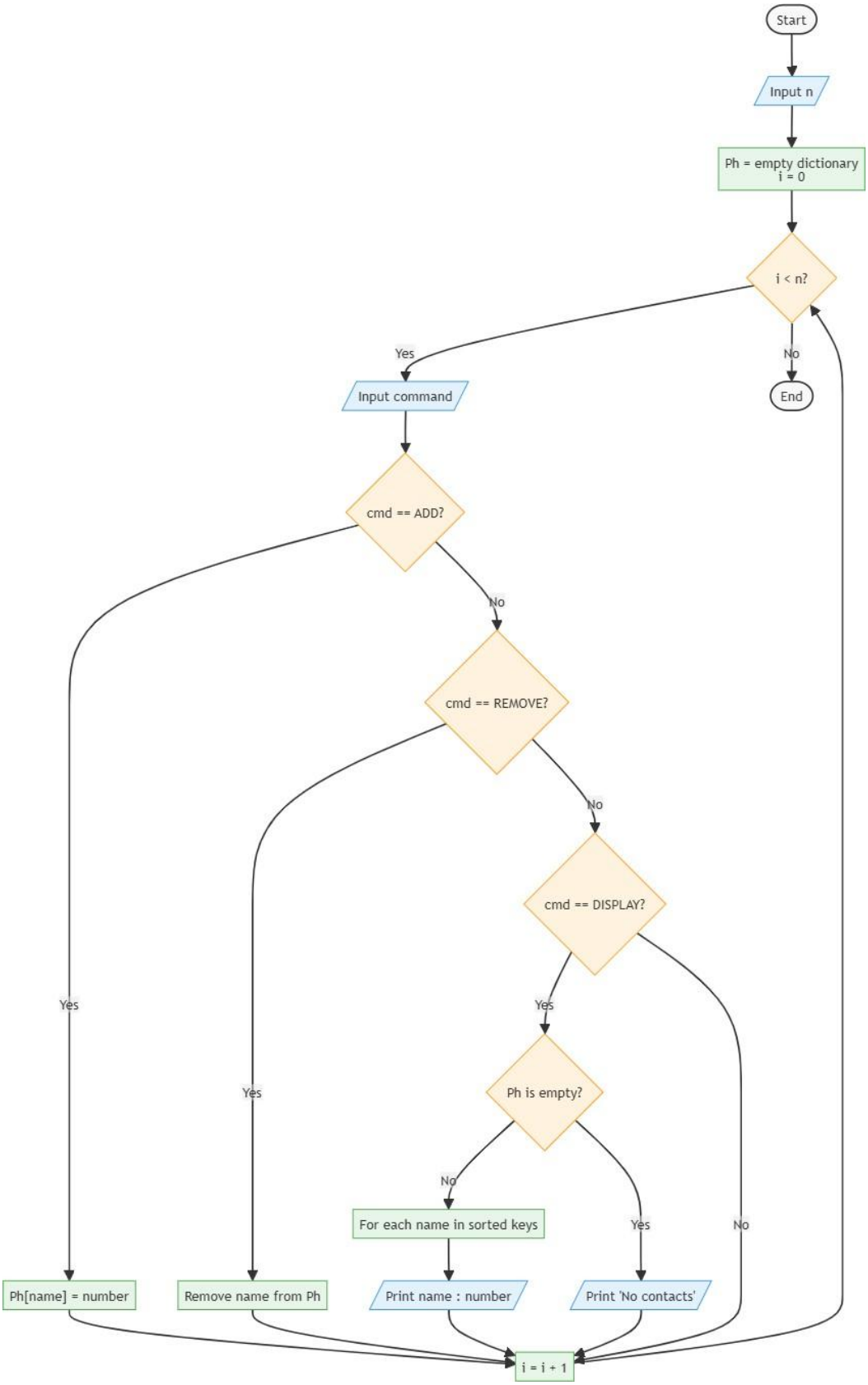
1. Aim

To design and implement a Python program to manage a phone number database using a dictionary. The dictionary stores contact names as keys and phone numbers as values. The program supports three operations: adding a contact (or updating if it exists), removing a contact (ignoring if it doesn't exist), and displaying all contacts in alphabetical order by name.

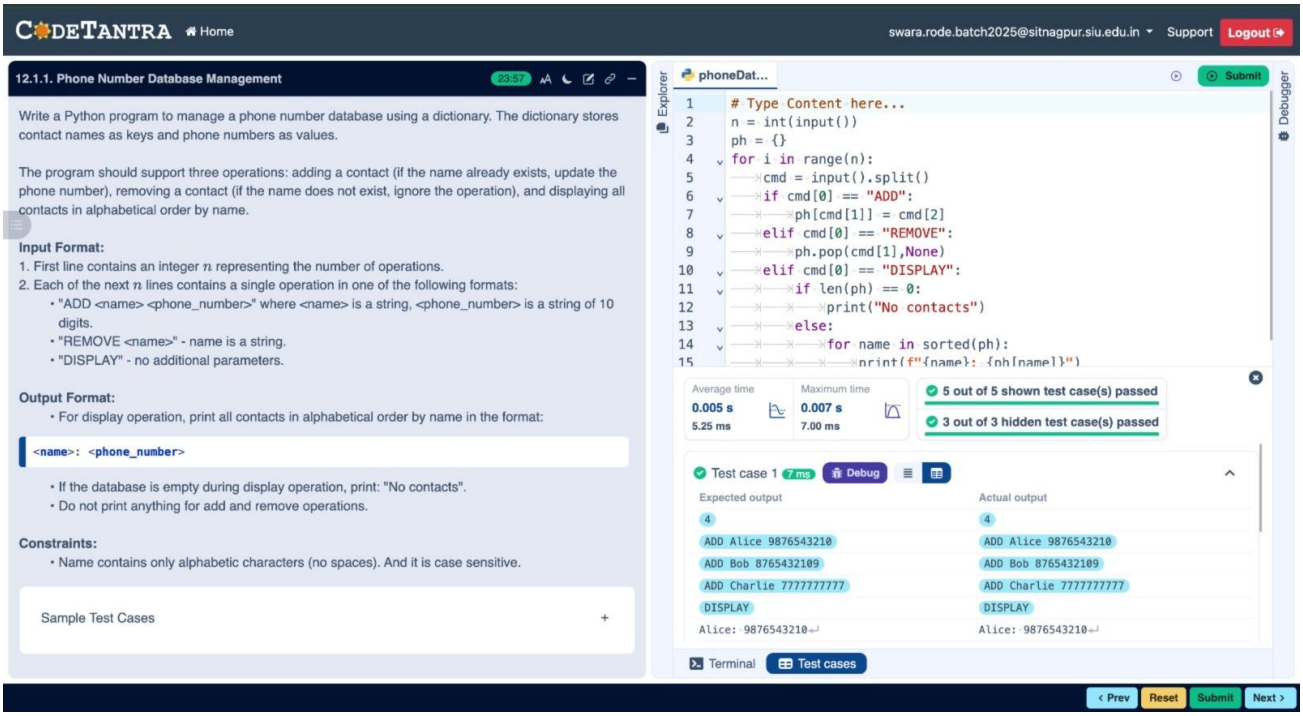
2. Pseudocode

1. **START**
2. **READ** an integer n representing the total number of operations to perform.
3. **INITIALIZE** an empty dictionary named Ph.
4. **FOR** i from 1 to n:
    - **READ** the operation command and parameters as a list cmd.
    - **IF** cmd[0] is "ADD":
        - Add or update the contact: Ph[cmd[1]] = cmd[2].
    - **ELSE IF** cmd[0] is "REMOVE":
        - Remove the contact if it exists using Ph.pop(cmd[1], None).
    - **ELSE IF** cmd[0] is "DISPLAY":
        - **IF** the dictionary Ph is empty:
            - **PRINT** "No contacts".
        - **ELSE**:
            - **SORT** the dictionary keys (names) alphabetically.
            - **FOR** each name in the sorted keys:
                - **PRINT** the contact in the format {name}: {phone_number}.
5. **END**

3. Flowchart


Start
Input n
Ph = empty dictionary
i = 0
i < n?
Yes
No
End
Input command
cmd == ADD?
No
cmd == REMOVE?
No
cmd == DISPLAY?
Yes
Yes
Yes
Ph is empty?
No
Yes
No
For each name in sorted keys
Ph[name] = number
Remove name from Ph
Print name : number
Print 'No contacts'
i = i + 1

4. Experiment Screenshot



Experiment 13.1.1: Fibonacci Sequence using Modules

1. Aim

To design and implement a Python program using modular programming to generate the Fibonacci sequence up to a given maximum value. The solution involves creating a separate module for the sequence generation logic and a main program for user interaction and validation.

2. Pseudocode

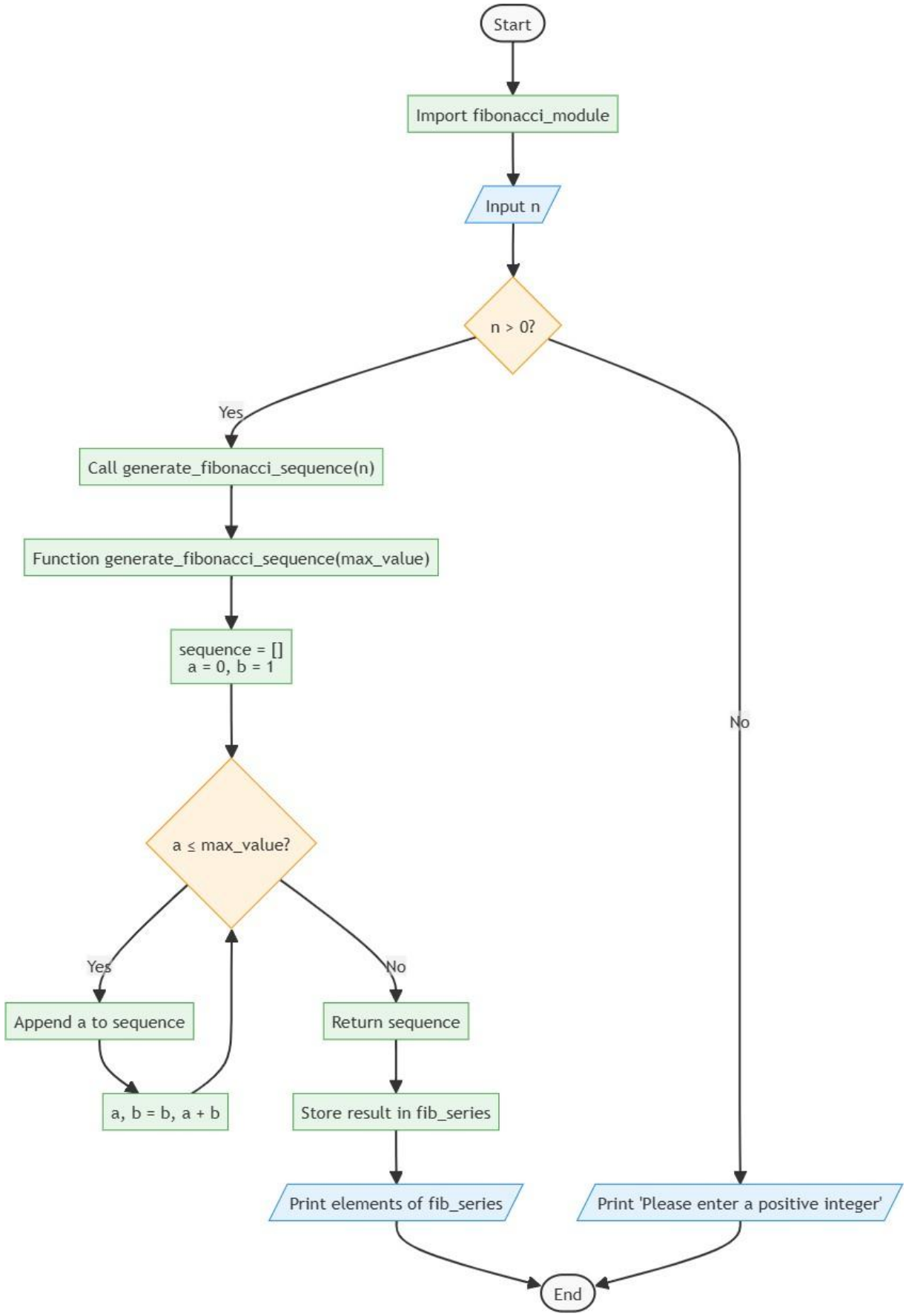
Module: fibonacci_module.py

1. **DEFINE** function generate_fibonacci_sequence(max_value):
   - **INITIALIZE** an empty list sequence.
   - **SET** $a = 0$ and $b = 1$.
   - **WHILE** $a \leq max_value$:
     - **APPEND** a to the sequence.
     - **UPDATE** a and b ($a = b, b = a + b$).
   - **RETURN** the sequence.

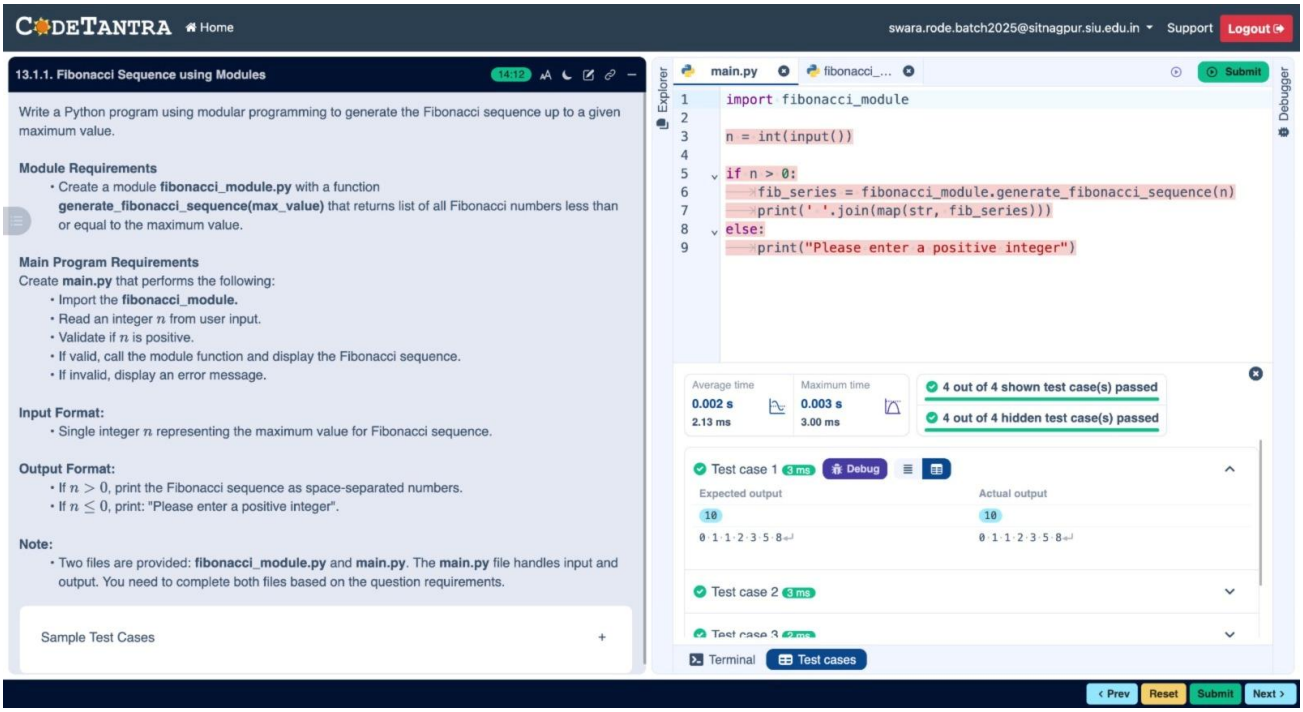
Main Program: main.py

1. **START**
2. **IMPORT** fibonacci_module.
3. **READ** an integer n from user input.
4. **IF** $n > 0$:
   - **CALL** fibonacci_module.generate_fibonacci_sequence(n) and store the result.
   - **PRINT** the Fibonacci sequence as space-separated numbers.
5. **ELSE**:
   - **PRINT** "Please enter a positive integer".
6. **END**

3. Flowchart


Start
Import fibonacci_module
Input n
n > 0?
Yes
No
Call generate_fibonacci_sequence(n)
Function generate_fibonacci_sequence(max_value)
sequence = []
a = 0, b = 1
a ≤ max_value?
Yes
No
Append a to sequence
a, b = b, a + b
Return sequence
Store result in fib_series
Print elements of fib_series
Print 'Please enter a positive integer'
End

4. Experiment Screenshot



Experiment 14.1.1: Complex Number Addition using Class

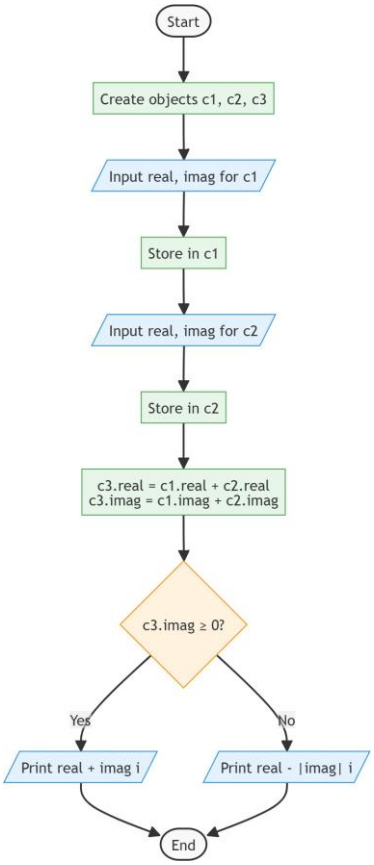
1. Aim

To design and implement a Python program using a class named **Complex** to add two complex numbers. The program involves using methods to initialize values from user input, compute the sum of two complex instances, and display the result in a standard complex number format ($a + bi$ or $a - bi$).

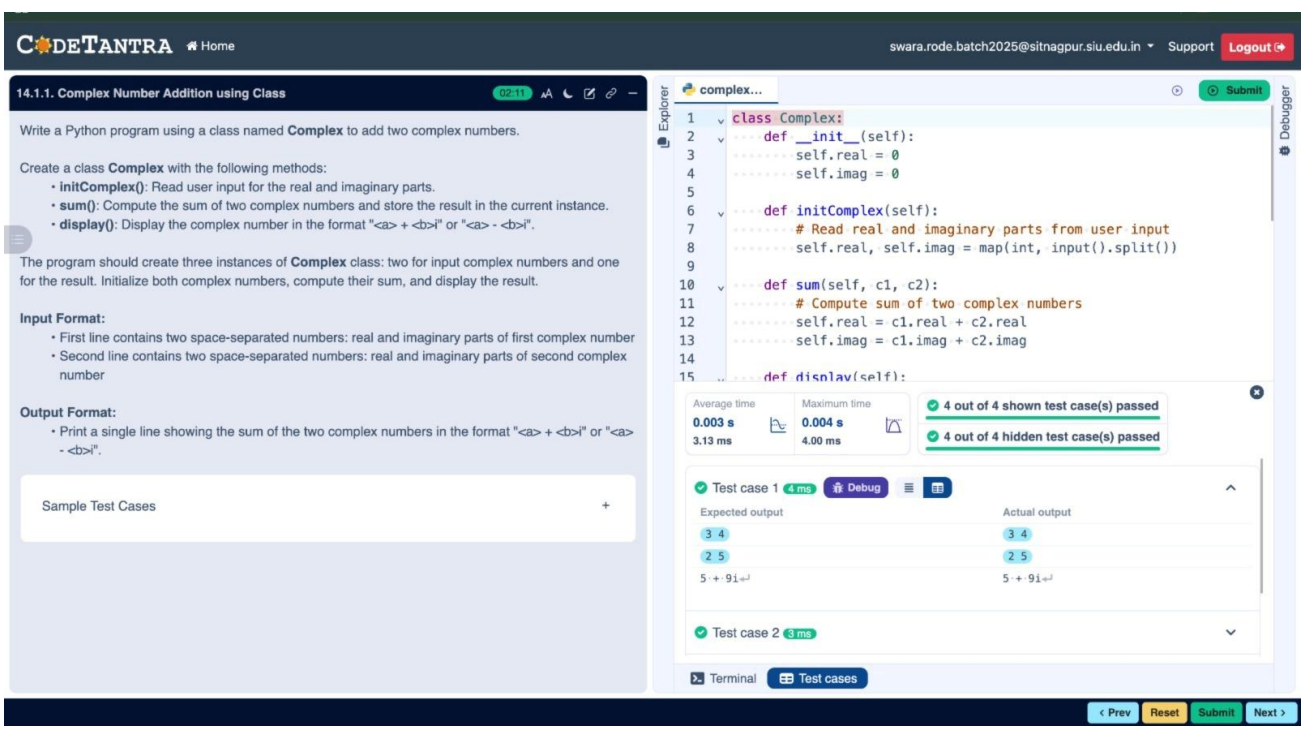
2. Pseudocode

1. **START**
2. **DEFINE** a class Complex:
   - **METHOD** initComplex(self):
     - **READ** two space-separated integers (real and imaginary parts) and **STORE** them in self.real and self.imag.
   - **METHOD** sum(self, c1, c2):
     - **SET** self.real = c1.real + c2.real.
     - **SET** self.imag = c1.imag + c2.imag.
   - **METHOD** display(self):
     - **IF** self.imag >= 0:
       - **PRINT** the complex number in the format: real + imag i.
     - **ELSE**:
       - **PRINT** the complex number in the format: real - abs(imag) i.
3. **MAIN PROGRAM**:
   - **CREATE** three instances of the Complex class: c1, c2, and c3.
   - **CALL** c1.initComplex() to read the first complex number.
   - **CALL** c2.initComplex() to read the second complex number.
   - **CALL** c3.sum(c1, c2) to compute the addition.
   - **CALL** c3.display() to output the result.
4. **END**

3. Flowchart



4. Experiment Screenshot



Experiment 15.1.1: Car Dealership Sales System

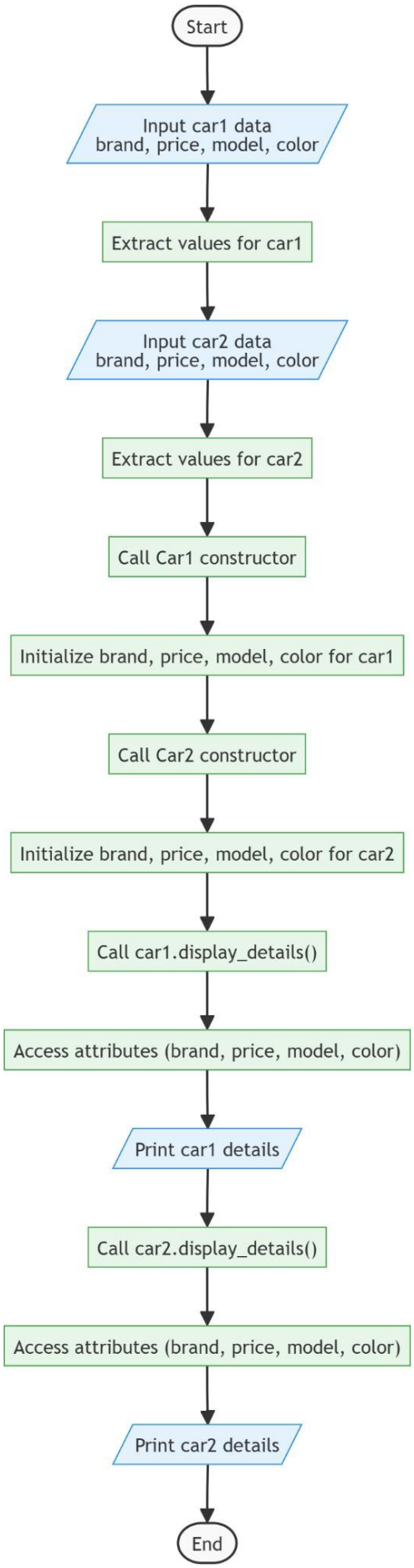
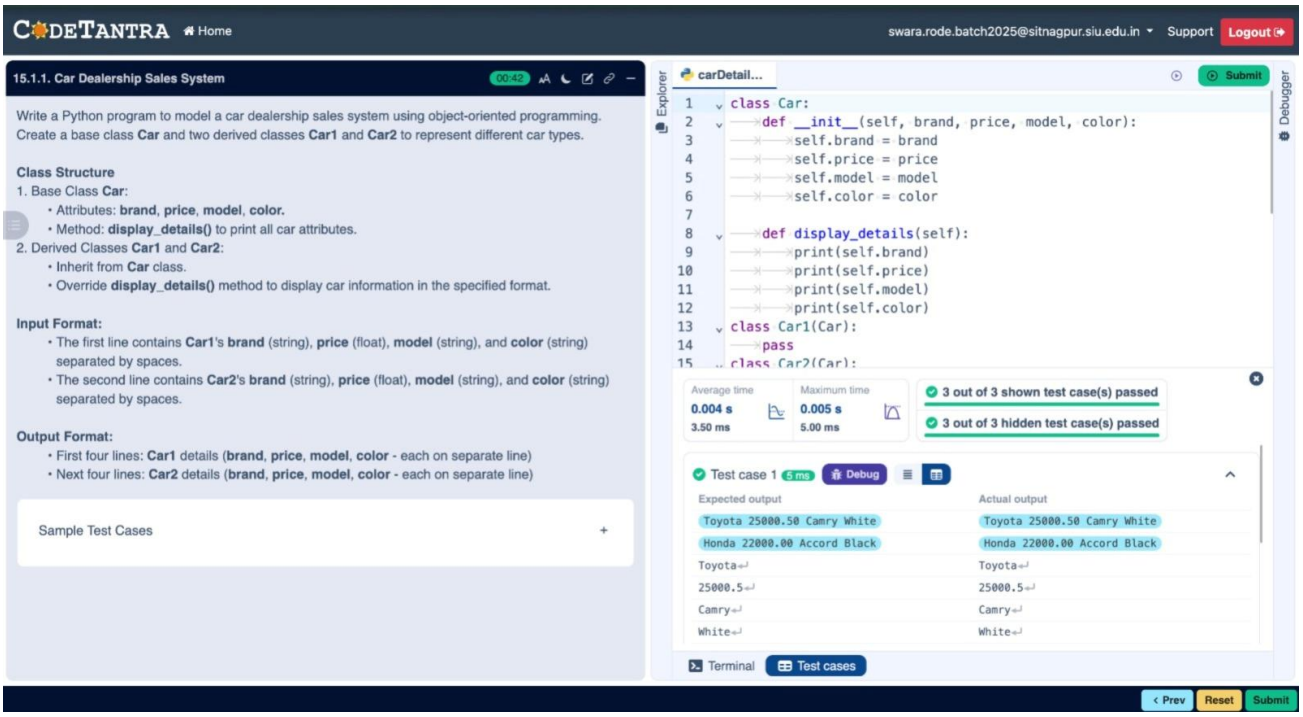
1. Aim

To design and implement a Python program to model a car dealership sales system using object-oriented programming. The task involves creating a base class Car and two derived classes Car1 and Car2 to represent different car types, demonstrating inheritance and method overriding.

2. Pseudocode

1. **START**
2. **DEFINE** a base class Car:
    - **METHOD** __init__(self, brand, price, model, color):
        - Initialize the attributes self.brand, self.price, self.model, and self.color.
    - **METHOD** display_details(self):
        - Print the attributes of the car.
3. **DEFINE** derived classes Car1 and Car2 inheriting from Car:
    - **OVERRIDE** the display_details(self) method in both classes to print car information (brand, price, model, and color) each on a separate line.
4. **MAIN PROGRAM**:
    - **READ** the input for Car1 (brand, price, model, color) from a single line.
    - **READ** the input for Car2 (brand, price, model, color) from a second line.
    - **INSTANTIATE** objects for Car1 and Car2 with the provided inputs.
    - **CALL** the display_details() method for the Car1 instance.
    - **CALL** the display_details() method for the Car2 instance.
5. **END**

3. Flowchart


Start
Input car1 data
brand, price, model, color
Extract values for car1
Input car2 data
brand, price, model, color
Extract values for car2
Call Car1 constructor
Initialize brand, price, model, color for car1
Call Car2 constructor
Initialize brand, price, model, color for car2
Call car1.display_details()
Access attributes (brand, price, model, color)
Print car1 details
Call car2.display_details()
Access attributes (brand, price, model, color)
Print car2 details
End

4. Experiment Screenshot